

Assignment 3 : Yolo Model

Name: Shrenik Deshmukh

PRN: 202301100035

Batch: DL4

1. Introduction

This project focuses on building a multi-task computer vision system using the Ultralytics YOLOv8 framework. The system is capable of performing multiple vision tasks including object detection, image classification, pose estimation, and oriented bounding box detection.

The main goal of this project is to understand how a single framework like YOLO can be used for different types of vision problems and to implement all tasks on a local machine without using any cloud platforms.

2. Objectives

- To implement object detection using YOLO models
- To perform image classification using YOLO classification models
- To detect human body keypoints using pose estimation
- To understand and implement oriented bounding boxes (OBB)
- To deploy and test all models locally

3. Tools and Technologies Used

- Python
- Ultralytics YOLOv8
- VS Code
- Roboflow (for datasets)
- OpenCV (internally used by YOLO)

4. Datasets Used

The datasets used in this project were obtained from Roboflow:

Object Detection Dataset

Car Detection Dataset

<https://universe.roboflow.com/drink-tracker/car-detection-pn2gk>

Image Classification Dataset

Dog vs Cat Dataset

<https://universe.roboflow.com/tae-7jslq/dog-vs-cat-5fmu0>

Oriented Bounding Box Dataset

Rotated Object Dataset

<https://universe.roboflow.com/dartcounter/rotated-idn1l/dataset/1>

Pose Estimation Dataset

COCO Pose Dataset (used via Ultralytics built-in configuration)

<https://docs.ultralytics.com/datasets/pose/>

5. Methodology

The project was implemented in multiple stages:

Step 1: Environment Setup

A Python virtual environment was created and the Ultralytics YOLO library was installed using pip. All work was done locally using VS Code.

Step 2: Data Preparation

Datasets were downloaded from Roboflow and extracted. The datasets were organized according to YOLO requirements:

- Detection datasets used YAML configuration
- Classification datasets used folder-based structure
- Pose dataset was used from built-in configuration

Step 3: Model Training

Object Detection

A YOLOv8 nano model was trained on the car detection dataset to detect objects using bounding boxes.

Image Classification

A YOLO classification model was trained to classify images into categories such as cats and dogs.

Pose Estimation

A YOLO pose model was trained to detect human keypoints such as joints and limbs.

Oriented Bounding Boxes (OBB)

An attempt was made to train a model for rotated bounding boxes using a Roboflow dataset.

Step 4: Model Evaluation

After training, each model was tested on validation images. The outputs included:

- Bounding boxes for detection
- Class labels for classification
- Keypoints for pose estimation
- Rotated boxes for OBB (where applicable)

Step 5: Deployment

All models were tested locally using command-line inference and webcam-based testing for pose estimation.

6. Results

- The object detection model successfully identified objects and drew bounding boxes with confidence scores.
- The classification model correctly predicted image classes such as cats and dogs.

- The pose estimation model detected human keypoints and generated skeleton structures.
- The OBB model demonstrated the concept of rotated bounding boxes, although dataset limitations were observed.

7. Challenges Faced

- Training on CPU was slow compared to GPU
- Dataset format issues (especially for classification and OBB)
- Path and folder structure errors
- OBB dataset compatibility issues

8. Conclusion

This project successfully demonstrates the implementation of a multi-task vision system using YOLOv8. It shows how a single framework can be used for multiple computer vision tasks such as detection, classification, and pose estimation.

The system was implemented entirely on a local machine without using any cloud platforms, fulfilling the assignment requirements.